

Advanced JUnit Testing Exercises

Advanced JUnit Testing Exercises

```
<dependency>
  <groupId>org.junit.jupiter</groupId>
  <artifactId>junit-jupiter</artifactId>
  <version>5.10.2</version>
  <scope>test</scope>
</dependency>
<dependency>
  <groupId>org.junit.platform</groupId>
  <artifactId>junit-platform-suite</artifactId>
  <version>1.10.2</version>
  <scope>test</scope>
</dependency>
```

Exercise 1: Parameterized Tests

EvenChecker.java

```
public class EvenChecker {
    public boolean isEven(int number) {
        return number % 2 == 0;
    }
}
```

EvenCheckerTest.java

```
import org.junit.jupiter.params.ParameterizedTest;
import org.junit.jupiter.params.provider.ValueSource;
import static org.junit.jupiter.api.Assertions.*;

public class EvenCheckerTest {

    EvenChecker checker = new EvenChecker();

    @ParameterizedTest
    @ValueSource(ints = {2, 4, 6, 8, 10})
    public void testIsEven(int number) {
        assertTrue(checker.isEven(number));
    }
}
```

Output:

```
Tests run: 5, Failures: 0, Errors: 0, Skipped: 0
```

Exercise 2: Test Suites and Categories

```
import org.junit.platform.suite.api.SelectClasses;
import org.junit.platform.suite.api.Suite;

@Suite
@SelectClasses({EvenCheckerTest.class, ExceptionThrowerTest.class})
public class AllTests {
}
```

Output:

```
Tests run: 6, Failures: 0, Errors: 0, Skipped: 0
```

Exercise 3: Test Execution Order

```
import org.junit.jupiter.api.*;
import static org.junit.jupiter.api.Assertions.*;

@TestMethodOrder(MethodOrderer.OrderAnnotation.class)
public class OrderedTests {

    @Test
    @Order(1)
    public void firstTest() {
        System.out.println("Running first test");
        assertTrue(true);
    }

    @Test
    @Order(2)
    public void secondTest() {
        System.out.println("Running second test");
        assertTrue(true);
    }

    @Test
    @Order(3)
    public void thirdTest() {
        System.out.println("Running third test");
        assertTrue(true);
    }
}
```

Output:

```
Running first test
Running second test
Running third test
Tests run: 3, Failures: 0, Errors: 0, Skipped: 0
```

Exercise 4: Exception Testing

ExceptionThrower.java

```
public class ExceptionThrower {
    public void throwException(boolean shouldThrow) {
        if (shouldThrow) {
            throw new IllegalArgumentException("Invalid input provided");
        }
    }
}
```

ExceptionThrowerTest.java

```
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

public class ExceptionThrowerTest {

    ExceptionThrower thrower = new ExceptionThrower();

    @Test
    public void testExceptionIsThrown() {
        Exception exception = assertThrows(IllegalArgumentException.class, () -> {
            thrower.throwException(true);
        });

        assertEquals("Invalid input provided", exception.getMessage());
    }
}
```

Output:

```
Tests run: 1, Failures: 0, Errors: 0, Skipped: 0
```

Exercise 5: Timeout and Performance Testing

PerformanceTester.java

```
public class PerformanceTester {
    public void performTask() throws InterruptedException {
        Thread.sleep(200); // simulate work
    }
}
```

PerformanceTesterTest.java

```
import org.junit.jupiter.api.Test;
import java.time.Duration;
import static org.junit.jupiter.api.Assertions.*;

public class PerformanceTesterTest {

    PerformanceTester tester = new PerformanceTester();

    @Test
    public void testTaskCompletesWithinTimeout() {
        assertTimeout(Duration.ofMillis(500), () -> {
            tester.performTask();
        });
    }
}
```

Output:

```
Tests run: 1, Failures: 0, Errors: 0, Skipped: 0
```